

# ONBOARDING\_LOG.md

---

## DevEx Audit & Onboarding Experience

---

### 1. Environment Audit

- **Dependency Management:** The `package.json` lists very old dependencies (e.g., `express ~4.13.1`, `pg-promise ^4.4.6`). The use of tilde `~` and caret `^` with such old major versions suggests significant potential for breaking changes if a fresh `npm install` pulls in patches that have drifted over years.
- **Containerization:**
  - The `Dockerfile` uses `node:19.4.0-bullseye-slim`, which is relatively modern compared to the dependencies. This mismatch (old code, new runtime) could introduce subtle bugs.
  - It installs `netcat`, likely for a health-check script that isn't immediately visible or is intended for manual debugging.
  - `docker-compose.yml` defines a standard `postgres` service but hardcodes passwords (`postgres:postgres`), which is classic non-prod behavior leaking into configuration.
- **Configuration:**
  - `config.js` is a major pain point. It switches configuration based on a `STAGE` environment variable.
  - **CRITICAL BLOCKER:** The default case in `config.js` falls back to `config_devel`, which points to a specific hardcoded IP `10.211.55.70`. A new developer trying to run this locally without setting `STAGE="LOCAL"` or `"DOCKER"` will experience immediate connection timeouts.

### 2. Friction Points (Time-to-Hello-World)

- **Hardcoded Secrets:** `app.js` contains a hardcoded session secret: `'ñasddfilhpaf78h78032h780g780fg780asg780dsbovcubuyvqy'`. This poses a security risk and makes environment rotation impossible without code changes.
- **Data Volatility:** `app.js` calls `init_db()` on **every startup**. `model/init_db.js` attempts to `CREATE TABLE`s. If they exist, it might fail or behave unpredictably depending on `pg-promise` error handling (it catches errors but swallows them in some places). This makes testing data persistence nearly impossible as the app tries to reset state on boot.

- **Implicit Dependencies:** The application logs to a file `app-custom.log` using `log4js`. If the container user doesn't have write permissions to the app directory (though `Dockerfile` sets `WORKDIR /app` and runs as root by default), this will crash.

### 3. Severity Assessment: HIGH

- **Justification:** The application is not "clone-and-run" friendly. The default configuration points to an unreachable private IP address ( `10.211.55.70` ). The aggressive database initialization logic risks data loss or startup crashes. The dependency tree is severely outdated, inviting security vulnerabilities and compatibility issues with modern Node.js versions.

### 4. Quick Wins

1. **Fix Config Defaults:** Change `config.js` default case to use `127.0.0.1` or throw a clear error if `STAGE` is missing, rather than failing silently on a timeout to a private IP.
2. **Externalize Secrets:** Move the session secret and database credentials into a `.env` file (using `dotenv`) and remove hardcoded strings from `app.js` and `docker-compose.yml`.
3. **Idempotent Database Init:** Modify `model/init_db.js` to check if tables exist before attempting creation, or move seeding logic to a separate script ( `npm run seed` ) rather than running it inside the main application boot loop.

---

## Critical Issues Running the Project

---

### 5. Execution Blockers (Based on Code Analysis)

#### 5.1. Static IPs and Unreachable Configuration

**Problem:** The `config.js` file contains three configurations with hardcoded static IPs:

```
var config_local = {
  "db": { "server": "postgres://postgres:postgres@127.0.0.1", ... }
}
var config_devel = {
  "db": { "server": "postgres://postgres:postgres@10.211.55.70", ... } // Unreachable private IP
}
var config_docker = {
  "db": { "server": "postgres://postgres:postgres@postgres_db", ... }
}
```

**Impact:**

- The default case falls back to `config_devel` with IP `10.211.55.70` (likely a VM or container from the original developer).
- Without setting `STAGE="LOCAL"` or `STAGE="DOCKER"`, the application attempts to connect to an unreachable IP.
- The resulting error is a generic timeout with no clear message, making diagnosis difficult.

**Solution Applied:**

- Set the environment variable `STAGE="DOCKER"` in `docker-compose.yml` to force the use of `config_docker`.
- For local execution (without Docker), set `STAGE="LOCAL"` before starting the application.

## 5.2. Inverted Logic in Database Initialization

**CRITICAL Problem:** The `model/init_db.js` file has the user insertion logic **completely inverted**:

```
db.one('CREATE TABLE users(...)')
  .then(function () {
    // TABLE CREATED SUCCESSFULLY → DOES NOTHING (empty users!)
  })
  .catch(function (err) {
    // TABLE ALREADY EXISTS (error) → TRIES TO INSERT USERS
    var users = dummy.users;
    for (var i = 0; i < users.length; i++) {
      db.one('INSERT INTO users(name, password) values($1, $2)', [u.username, u.password]);
    }
  });
```

**Why this breaks everything:**

1. **First startup (empty DB):** `CREATE TABLE` succeeds → `.then()` executes → **NO users inserted** → empty table.
2. **Second startup (table exists):** `CREATE TABLE` fails → `.catch()` executes → **NOW users are inserted**.
3. **Result:** Login NEVER works on first startup because there are no users in the database.

**Evidence in code:**

- `dummy.js` defines default credentials: `admin/admin` and `roberto/asdfpiuw981`.
- `routes/login.js` calls `auth(user, password)` which executes an SQL query.
- `model/auth.js` searches the `users` table, but it's empty on first startup.

**Solution Applied:**

1. **Option A (temporary):** Start the application twice so users are inserted on the second startup.
2. **Option B (correct):** Invert the logic in `init_db.js`:

```
javascript db.one('CREATE TABLE users(...)') .then(function () { // TABLE CREATED → INSERT
```

```
USERS var users = dummy.users; for (var i = 0; i < users.length; i++)
{ ... } }) .catch(function (err) { // Table already exists, do nothing or log });
```

3. **Option C (best practice):** Use `CREATE TABLE IF NOT EXISTS` + `INSERT ... ON CONFLICT DO NOTHING` to avoid errors.

### 5.3. Broken and Vulnerable Login Service

**Problem:** The `model/auth.js` file builds SQL queries through direct string concatenation:

```
function do_auth(username, password) {
  var q = "SELECT * FROM users WHERE name = '" + username + "' AND password = '" + password + "';";
  return db.one(q);
}
```

#### Impacts:

1. **Critical SQL Injection:** An attacker can inject `admin' OR '1'='1` as username and bypass authentication.
2. **Credentials issue connection:** If the `users` table is empty (see point 5.2), **no credentials work**, not even `admin/admin`.
3. **Poor error handling:** `db.one(q)` throws an exception if it doesn't find exactly 1 result, causing a generic redirect to `/login?error=<message>`.

#### Evidence of the complete broken flow:

```
User attempts login (admin/admin)
↓
routes/login.js → auth(user, password)
↓
model/auth.js → SELECT * FROM users WHERE name = 'admin' AND password = 'admin'
↓
NO RESULTS (empty table due to inverted init_db.js)
↓
.catch() → res.redirect("/login?returnurl=" + returnUrl + "&error=" + err.message)
↓
User sees: "No data returned from the query"
```

#### Solution Applied:

1. Fix `init_db.js` first so users exist in the DB.
2. For quick testing: Manually connect to PostgreSQL and insert users:
 

```
sql INSERT INTO users(name, password) VALUES ('admin', 'admin'); INSERT INTO users(name, password) VALUES ('roberto', 'asdfpiuw981');
```
3. Use `STAGE="DOCKER"` to ensure database connectivity.

## 6. Procedure to Run Successfully

Steps applied to achieve functional execution:

1. Set environment variable:

```
bash export STAGE="DOCKER" # Linux/Mac $env:STAGE="DOCKER" # PowerShell Windows
```

2. Start services with Docker Compose:

```
bash docker-compose up --build
```

3. Wait for PostgreSQL initialization (check logs until you see `database system is ready to accept connections`).

4. Restart application container (so `init_db.js` executes the `.catch()` and inserts users):

```
bash docker-compose restart app
```

5. Verify inserted users (optional):

```
bash docker exec -it <postgres_container> psql -U postgres -d vulnerablenode -c "SELECT * FROM users;"
```

6. Access the application: `http://localhost:3000/login`

- Working credentials: `admin` / `admin` or `roberto` / `asdfpiuw981`

---

## 7. Confirmed Vulnerabilities (Code Analysis Only)

1. SQL Injection in authentication ([model/auth.js](#))
2. Hardcoded session secret ([app.js](#))
3. Database credentials in plaintext ([config.js](#))
4. Inverted initialization logic causing DoS on first startup ([model/init\\_db.js](#))
5. No CSRF protection (no CSRF tokens in login/purchase forms)
6. Potential Log Injection ([routes/login.js](#) - direct concatenation in logs)